Migrating CakeML Type Soundness from HOL4 to Lean with LLM Agents: An Experience Report

***Abstract*—Large language model (LLM) agents can translate formal developments between proof assistants, but a translation can compile, and even support checked proofs, while it differs in meaning from its source. We report our experience migrating the definitions and theorem statements that CakeML's type soundness theorem depends on from HOL4 to Lean 4. An agent translated and repaired the development, and a second model compared the translation with the source, with human adjudication introduced after incorrect repairs. Proving agents then attempted the theorems, with each proof audited through all its dependencies. Scripted structural checks exposed placeholders that compile and definitions without bodies. The second model found semantic errors but also proposed wrong repairs, and most of its early findings were not defects. Proof attempts exposed false statements and vacuous translations of nontrivial source theorems that review had missed. After repairs to statements and definitions, the automated prover Aristotle proved 267 of the 269 theorems with their statements unchanged, the main theorem included, using only Lean's standard axioms, and the other two translated statements were refuted. A later review of the translated library still found incorrect word-shift operations in the main theorem's dependencies, so a checked soundness proof can coexist with an unfaithful translation. From this experience we distil a migration workflow combining structural checks, human adjudication of model findings, proof audits through all dependencies, and explicit review of library dependencies, with proof-discovered defects returned to repair. We identify the claims each check supports and the semantic gaps that remain.**

***Index Terms*—proof migration, HOL4, Lean, CakeML, large language models, theorem proving**

I. Introduction

Interactive theorem provers such as HOL4, Isabelle/HOL, Rocq, and Lean [1], [2], [3], [4] give strong guarantees because a small kernel checks every proof [5]. Developments built on them, such as the verified compiler CakeML [6], represent years of proof engineering [7], and their definitions, libraries, notation, and automation all depend on the source system.

Lean 4 is an attractive target for such a development: it is both a prover and a programming language, it has a large shared library, Mathlib [8], and most recent machine learning provers work in it [9], [10], so a development moved to Lean can be read, extended, and proved with these tools.

In a prover, definitions and theorem statements are source code that a build checks, so migrating a development resembles migrating software. Large language models (LLMs) already translate and migrate ordinary software [11], [12], and LLM agents can read a repository, edit files, run a build, and respond to errors [13], [14], [15]. For software, tests show whether the migrated code still behaves as before. For a formal development, a successful build shows less than it seems, because Lean accepts placeholders. The keyword `sorry` closes any proof without proving it, and we call what it closes *admitted*. A theorem can be restated as `True`, which holds trivially. A definition can be declared `opaque`, which gives a name that no proof can unfold, so the intended implementation can be left out. A recursive function can be declared `partial`, which skips its termination proof but hides its equations from every later proof. Each placeholder compiles while saying less than the source, so a translation that builds can still be wrong.

This paper reports on one such migration. The source is the HOL4 development behind CakeML's type soundness theorem [6], [16], reduced to the 20 theory scripts that the theorem depends on. The target is a Lean project that keeps definition bodies and theorem statements as readable source. We translated definitions and statements only and left every proof as `sorry`, because an agent that must also prove a statement can make the proof easier by weakening the statement. We had no route from Lean back to HOL4 for a mechanical check such as that of lf-lean [17], so our question is what evidence is needed to assess an LLM-generated migration, and where each available check fails: the build, scripted scans, review by a second model, and proof attempts.

The process grew through the failures we report: human experts began to decide on review findings only in review round 4, after wrong findings had been applied, and proving, planned as a measurement of how much an LLM can prove, became a check once it exposed defects that review had missed. Section III gives the workflow we now recommend and the process as it ran, and Section VIII the lessons behind the recommendation. An earlier account of the case appears in a master's dissertation [18]; this paper adds post hoc checks, made after the study, of every review finding, an audit of each proof through its dependencies, the repairs, and a second proving round; these also correct that account.

Each check after the build exposed defects that the earlier checks had missed (Table I). The checks ran in sequence on a changing translation, so this is an observation, not a controlled comparison. Post hoc scripts found statements reduced to `True` and missing bodies that the build accepted. The second model, which we call the validator, found semantic drift, changes of meaning that still compile, but before acceptance for proving it proposed more wrong changes than right ones. Proof attempts showed statements that review had approved to be vacuous or false, but missed wrong word shifts, under which the main theorem still holds, in the library layer; a review of that layer found them.

TABLE I

WHAT EACH CHECK EXPOSED AND MISSED, DETAILED IN SECTIONS IV TO VII. R: REVIEW ROUND. WHY A CHECK MISSED A DEFECT: ^{i} OUTSIDE WHAT THE CHECK, AS APPLIED, CAN ESTABLISH, ^{s} SUPPLIED TO THE CHECK BUT NOT IDENTIFIED, ^{n} THE INPUT NEEDED WAS NOT SUPPLIED, ^{x} THE CHECK WAS NOT RUN ON THAT PART, ^{u} LEFT UNPROVED WITHOUT A DIAGNOSIS.

Check	Exposed	Missed
Build	Type and import errors	Every defect below^{i}
Human inspection (R1, R2)	Invented interface, `True` statements	Library stubs^{x}
Structural scan (post hoc)	`True` statements, missing bodies, `partial`, admitted instances	Other semantic discrepancies^{i}
Validator (R3 to R6)	Missing operators, dropped domain condition, comparator type, free-variable grouping (R5)	String indexing^{n}, equality without laws^{i}, vacuous theorems^{s}. R3 findings introduced the grouping and binding order defects
Proofs, pilot and round 1	Missing bodies, equality, string indexing, vacuous theorems	Sparse tree keys, set cardinality^{u}
Proofs, round 2	Sparse tree keys, set cardinality	Word shifts, fused multiply-add^{i}
Library validator (R7)	Sparse tree keys, set cardinality, word shifts, fused multiply-add	Finite map finiteness^{s}

The paper makes three contributions. (1) A migration case whose failures can be checked: every validator finding of rounds 3 and 4 checked against HOL4, every defect confirmed by a refutation or a check in Lean, and an artifact that recomputes the numbers (Sections V to VII). (2) Evidence on what the build, structural checks, review by a second model, and checked proofs each establish about a translation and where each stops, including a proof of the main theorem with only Lean's standard axioms over library operations that still differ from HOL4. The proof counts of two rounds of five proving workflows are supporting evidence for these findings. (3) A recommended workflow with the lessons behind it and the uncertainties that remain (Fig. 1, Section VIII). Section II describes the case and Section III the process. Sections IV to VII present one finding per check: the build, review, proof attempts, and what a checked proof leaves open. Each says which decision rested on the check, what happened, how we established it, and what it changed.

II. THE CASE: CAKEML TYPE SOUNDNESS

CakeML is a verified implementation of an ML dialect whose syntax, semantics, compiler, and metatheory are formalised in HOL4 [6], [19]. Tan et al. first proved type soundness [16]; the development we migrate is its restatement for the functional big step semantics, a clocked evaluator [20]. Its main theorem (Fig. 4) states that if a program is well typed, and the state and environment satisfy an invariant that relates runtime values to their types, then no run of the evaluator stops with a runtime type error. It says nothing about which value an operation computes: a primitive that returns a word of the right type but the wrong value, such as a wrong shift, leaves the theorem provable. The proof depends on the abstract syntax, nested namespaces, runtime values and stores, the evaluator, typing relations, substitution and weakening lemmas, and invariants that connect runtime values to their types.

Scope. We migrated only the dependency closure of the type soundness theorem, so that every migrated item serves the main theorem and the task stays small enough to review in full. Starting from a standalone package of 47 files and about 29,000 lines, we used the dependency information that the HOL4 kernel records for each theorem to keep only what the main theorems transitively depend on, and HOL4 rebuilt the package after every step. The final package has 20 theory scripts, a preamble, and a build file, with 11,302 lines of SML. Not counting blocks that are commented out, they contain 255 definition blocks, 4 blocks of inductive relations, 56 datatypes, 24 type abbreviations, and 280 theorems.

Target. The Lean project pairs each theory script with one Lean file, so that review can compare file pairs, and adds a compatibility layer, `HOL4Prelude.lean`, for HOL4 library notions such as finite maps, sets, words, and floating point. The baseline, the state frozen for proving (Section III), has 8,352 lines including embedded HOL4 source comments. Its 382 definitions and 269 theorems form the canonical manifest that all measurements use. The prelude holds 112 of the definitions. Inductive relations, inductive datatypes, structures, instances, and type abbreviations are outside the manifest.

The manifest comes from the target, so we compared its theorems with the source. Lean generates its own version of 6 of the 280 HOL4 theorems, the equation and induction theorems of the evaluator. The baseline states 266 of the remaining 274, 9 of them under new names; the 8 absent are 7 local helpers and 1 exported lemma, and the baseline adds 3 theorems with no counterpart. We therefore report three outcomes separately: source coverage, the HOL4 theorems that the baseline states (266 of 274); structural completeness, the 651 manifest obligations that pass the counting rules of Section IV; and proof completion, the proved theorems among the 269 of the manifest. None implies another, and none shows that the translation means what HOL4 means.

Faithfulness. We judge the translation at three levels. *Representations:* each HOL4 value has a corresponding Lean value; a Lean type may hold extra values, and a representation may need a well-formedness premise, under which every statement about it is read. *Operations:* on corresponding inputs each definition computes the corresponding result; where HOL4 leaves a result unspecified, our policy lets the Lean definition fix one (Table II). *Statements:* each theorem means the same, and in particular no translated statement may become false or trivial. Any other difference is a defect. For example, HOL4's character type has 256 values, while Lean's `Char` holds every Unicode scalar value; a HOL4 character with code n corresponds to the Lean character with code n. The translation states no range premise, which the statements allow: they generalise over the extra characters and remain provable, as the proofs of Section VI-E show. Whether the string operations agree with HOL4 on represented values is a separate obligation, not proved here: it was inspected by

TABLE II
HOL4 AND LEAN DIFFERENCES THAT SHAPED THE TRANSLATION, AS HANDLED IN THE FROZEN BASELINE. THE PROOF STAGE EXPOSED THE ROWS MARKED AS A DEFECT, AND THE REPAIR (SECTION VI-D) CHANGED THEM.

HOL4	Handling in the Lean target
Total functions whose equations may leave constructors unspecified	A fixed default value, which keeps the source type and refines the unspecified value
Recursive definitions come with termination proofs	Ordinary `def` with an explicit termination obligation, `partial def` forbidden
One Boolean type for data and formulas	`Bool` for executable predicates, `Prop` for relations and statements
Nested datatypes through association lists	Nested occurrences written as `List` pairs to satisfy Lean's rules for nested inductive types
Lawful equality on every type	Boolean equality (`BEq`) without the law `LawfulBEq` that ties it to equality (a defect)
Strings as lists of 8-bit characters	Lean `String` of Unicode characters, indexed by byte positions (a defect)
IEEE binary64 as a HOL4 theory	Lean's `Float`, opaque to the kernel
Infix operators with HOL4 precedences	Lean operators with their own precedences
Theorems that HOL4 computes by rewriting	Stated as $P \leftrightarrow P$ (a defect); the repair writes out the computed case expansion

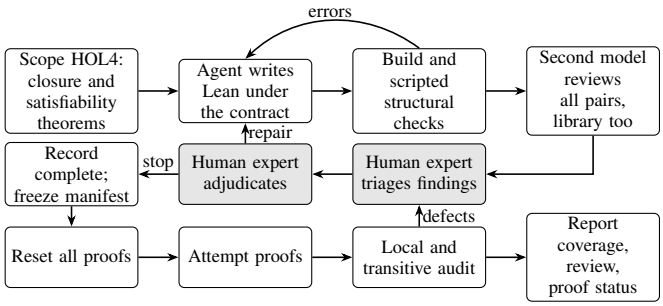


Fig. 1. The recommended workflow (Section VIII). Shaded steps are human decisions. Table III lists where the study deviated from it.

review and spot-checked in Lean after the repair of `strsub` (Section VI-D). The sparse trees of Section VI-E are the case where a premise was needed and missing.

Mismatches. Table II lists the differences between the systems that shaped the translation. Termination is a typical case: HOL4 derives the recursion equations of a function from a well-founded relation [21], while Lean 4 has its own termination obligations [22].

III. STUDY PROCESS

Figure 1 shows the workflow we recommend after the study, and Table III the study as it ran in seven review rounds, with each step's deviation from that workflow. We assembled the workflow across the rounds; no round followed all of it, and each deviation led to a lesson of Section VIII. Its principle is that each claim needs evidence that can support it: dependency analysis for scope, the Lean build for well-formedness, structural checks for missing or vacuous declarations, paired source comparison for semantic drift, human decisions for repairs, and fixed declaration names for comparisons between repository states.

TABLE III
THE STUDY AS IT RAN, AND WHERE EACH STEP DEVIATED FROM FIG. 1. DATES ARE IN 2026; ROUNDS ARE REVIEW ROUNDS (TABLE V). L: THE LESSON IT LED TO.

When and what	Deviation	L
13 to 26 Feb: translation, rounds 1 and 2	Human expert inspection in place of the second model; contract checked by hand, not by script	L1
26 Mar: round 3	No triage or adjudication; every finding went to the agent	L2
1 to 8 Apr: round 4, acceptance	Triage withheld a confirmed finding; two pairs had no verdict; the library was never paired with a source	L2, L3
23 to 29 Apr: pilot, definition pass, freeze	Pilot ran on a translation with 61 definitions without bodies	L1
May: proving round 1	Defects that proving exposed returned to review only in October	L4
5 Oct: rounds 5 and 6, repairs	The library was still not paired with a source	L3
Oct: proving round 2	Prover repaired two statements; excluded from the accepted proof totals, review pending	L4
8 to 15 Oct: scripted scan, round 7	Scan and library review written and run after the study; word shifts not yet repaired	L3

Translation protocol given to the agent (abridged)

```
Goal: translate HOL4 theories to Lean 4, keep context.
Types: nonrecursive -> structure, recursive -> inductive.
Names: prefix Lean reserved words with cml_ (id -> cml_id).
Libraries: use Lean or Mathlib equivalents if they exist.
Imports: import ancestor theories, reuse their names.
Do not invent definitions or functions.
Output theorem statements only, closed with := by sorry.
Never simplify a theorem statement to True.
sorry may skip a proof, never a definition body.
No partial def: use def, leave only termination as sorry.
Record kind, name, Lean, HOL4, theory, ancestors per item.
```

Fig. 2. The translation contract, abridged from the protocol used in the project. Rules on imports, invented definitions, `True` statements, definition bodies, and file records were added after round 1, and the `partial def` rule in round 3.

A. Translation contract

The agent received a short translation protocol that grew as failures appeared (Fig. 2). The contract allowed Mathlib, but the agent wrote its own prelude, and no module of the final baseline imports Mathlib. Theorem statements must be translated in full and closed with `sorry`, and a theorem may not be weakened to `True`. A `sorry` may close a proof and may discharge a termination obligation in a `decreasing_by` clause, but it may not replace a definition body, and `partial def` is forbidden.

B. Agentic generation and repair from compiler feedback

The translation agent was Claude Code [15] running Claude Opus 4.6 in rounds 1 to 3; every later Claude Code run used Claude Opus 4.7. It worked on the repository directly: it read the HOL4 sources, wrote Lean files, ran `lake build`, and repaired errors bottom up, from the prelude to the soundness proof, the feedback loop of self-debugging methods [23] with the compiler as the source of feedback. This mattered early: a pipeline that called Gemini once per declaration failed on a nested datatype that Lean rejected, while the agent found an accepted form from the error.

After each repair the agent rebuilt the whole project. At the milestones of Table IV, human experts and the agent checked the contract by hand: canonical names present, no theorem stated as `True`, and no definition body that is directly `sorry` or `opaque`; the `partial def` check came in round 3, when a human expert found functions that later proofs could not unfold.

C. Paired review and adjudication

A compiling and structurally complete project can still disagree with its source. The validator therefore compared each of the 20 HOL4 theory scripts with its Lean file. It used Gemini 2.5 Pro [24], a model from a different family than the translation agent's, so that the generator's assumptions were not the only semantic signal. The validator received the full text of both files in one prompt at temperature 0.1, with the instruction to find genuine translation errors only and to ignore missing or simplified proofs unless they change meaning. For each file pair it answered `RIGHT` or listed each error with its kind, severity, evidence from both files, and an explanation. A `RIGHT` answer means only that the validator reported no issue within the two files it was given. After the proof study, rounds 5 to 7 reran the round 4 prompt with newer fast models, Gemini 3.8 Flash with high reasoning in rounds 5 and 6 [25] at the same temperature, and Claude Haiku 5.5 with no temperature set in round 7 [26].

The handling of validator output changed between rounds. In round 3 every finding went to the agent with the source evidence, in sessions run by a human expert, and the agent decided which findings to apply. Because the agent had applied wrong findings in round 3 (Section V), from round 4 on human experts decided: they triaged the findings, and findings that touched types, assumptions, totality, or representation choices were escalated for adjudication. The loop stops in the first round in which no finding survives adjudication. This happened in round 4, where no escalated finding survived, although a confirmed finding that triage had not escalated remained unresolved (Section V). The translation was then accepted and went to the proof stage, which exposed defects and sent it back to review. In round 5 a human expert triaged and adjudicated the findings, and in round 6 the validator reported no finding, so the loop stopped again.

Roles. Besides the agents, human experts wrote the contract, inspected the outputs, ran the validator and the agents, triaged and adjudicated the findings, and advised on representations. The scripted checks and the validator are cheap: in rounds 5 and 6 a validator run took about ten minutes, and a round of three runs cost about US$2 of model use. Triage, adjudication, and repair review need a reader of both HOL4 and Lean, and we did not measure that time; the record holds the dates of Tables IV and V and the session logs.

D. Freeze and proof measures

After acceptance and a pass that supplied missing definition bodies (Section IV), the 382 definition names and 269 theorem names of the baseline became the canonical manifest. All theorem proofs were reset to `sorry` while definitions and statements stayed fixed, so that every proving workflow starts from the same task. When a proof attempt exposes a translation defect, the defect goes back to triage (Fig. 1), and proofs are counted again only on the repaired translation. We measure every proving result against the canonical names, so helper lemmas added by an agent do not enlarge the task, and canonical declarations that disappear count as unproved.

A placeholder count sees only the proof text, not the admissions that a proof reaches through its dependencies, so we use three measures. A theorem is *locally complete* when its canonical name is present with a statement other than `True` and its declaration contains no `sorry` or `admit`. A theorem is *audited* when, in addition, Lean's axiom collection over its full transitive dependency closure, statement included, finds only the standard axioms `propext`, `Classical.choice`, and `Quot.sound` [27]. The audit is the final measure. Between the two, a theorem is *complete modulo admissions* when it is locally complete and every admission in its transitive closure is one of the admitted definitions of the frozen baseline: the termination proofs that the contract allows and the two equality instances described in Section IV. This is a progress measure relative to the frozen baseline: it credits a proof while the definitions it rests on are still being repaired. It trusts an admitted termination proof provisionally, because HOL4 proves that the source function terminates, which carries over only if the translated function corresponds to it. An admitted instance instead leaves the comparison function unspecified, a translation defect (Table VIII). A proof that uses an admitted lemma fails this measure. The strict audit also fails proofs whose statement or proof reaches an admitted definition, so it measures the repair of definitions as well as proofs. We also compared the 103 declarations outside the manifest with their starting point at every proving endpoint, the final repository state of a run.

IV. The Build Accepted an Incomplete Specification

The translation was accepted for proving on a clean build and hand checks at milestones. The build had accepted placeholders, and the manual checks had missed some of them, so the first proving pilot ran on definitions without bodies. At the time, each discovery led to a repair pass; the scripted structural scan that gives the counts below was written after the study and replayed over every repository state.

The scan counts a definition as translated when its canonical name is present and its body is not `opaque`, not `partial`, and not directly `sorry`; a termination `sorry` keeps the body and is allowed. It counts a theorem statement when its canonical name is present and its proposition is not literally `True`. Table IV gives structural completeness at each repository state that changed the method.

The initial translation compiled and covered 291 obligations. The remaining 360 were 21 missing definitions, 40 missing theorems, and 299 placeholders that the compiler accepted: 229 theorems stated as `True` (35.2% of all obligations), 57

TABLE IV
STRUCTURAL COMPLETENESS OF THE 651 MANIFEST OBLIGATIONS AT EACH REPOSITORY STATE THAT CHANGED THE METHOD. DEF.: DEFINITIONS WITH BODIES (OF 382). STMT.: STATEMENTS OTHER THAN TRUE (OF 269).

Date	Stage	Def.	Stmt.	Complete
13 Feb	Initial agent translation	291	0	291 (44.7%)
19 Feb	Dependency repair	306	0	306 (47.0%)
19 Feb	Statement pass	307	244	551 (84.6%)
19 Feb	Definition audit	311	245	556 (85.4%)
26 Feb	Full stub scan	311	269	580 (89.1%)
26 Mar	Validator round	321	269	590 (90.6%)
29 Apr	Definition completion	382	269	651 (100%)

HOL4, which parses as $(A \cup B) \setminus C$

```
FV (Letrec defs b) = FV_defs defs ∪ FV b DIFF set (MAP (
     Short o FST) defs)
```

Lean before round 3, agrees with HOL4

```
| .Letrec defs b => (FV_defs defs ∪ FV b) \ names defs
```

After round 3, validator-induced regrouping

```
| .Letrec defs b => FV_defs defs ∪ (FV b \ names defs)
```

Fig. 3. Drift introduced through a wrong validator finding. HOL4 declares `UNION` and `DIFF` left associative at precedence 500, while Lean's \ binds tighter than ∪, so the HOL4 grouping needs the parentheses that the round 3 change removed. Both versions compile.

definitions with a direct `sorry` body, 10 definitions declared `partial`, and 3 `opaque` definitions.

A human inspection found theorems stated as `True` and led to the statement pass (306 to 551, Section V). After acceptance, a first proving pilot with Claude Code tested whether the statements could be proved at all. It completed 48 theorems locally, but many other goals depended on definitions without bodies. The scan, replayed on the accepted state, shows that 61 canonical definitions had none: 7 semantic primitives, among them `do_app`, which gives the semantics of every primitive operation, and 54 prelude definitions. The milestone checks had been manual and had looked only at theory files with a HOL4 partner, so they missed the prelude stubs, and the validator did not report the stubbed `do_app` because its prompt told it to ignore missing proofs. A definition pass, run by a human expert under the same contract, supplied all 61 bodies (590 to 651), after which the project built and the structural checks passed. The proofs were then reset and the baseline frozen.

The frozen baseline still contains two kinds of admitted obligations. Thirteen recursive definition groups, including the evaluator, pattern matching, and expression traversals, leave their termination proofs as `sorry`, which the contract allows. Two `BEq` instances, on runtime values and on environments, admit their whole comparison function as `sorry`. Instances fall outside the manifest, so the contract check missed them.

V. REVIEW FOUND DRIFT AND INTRODUCED IT

Acceptance rested on review by a second model: a translation that compiled and that the validator approved went to proving. The validator did find semantic drift that no build reveals, but before acceptance most of its findings were wrong, and wrong findings applied without review introduced drift. Wrong repairs noticed during round 3 led to human decisions on findings from round 4 on; the check of every round 3 and 4 finding against the HOL4 sources (Table VI) was made after the study. Table V summarises all seven rounds.

Rounds 1 and 2. A human expert's inspection of the first translation found problems that a build cannot report: a missing import, for which the agent had invented a replacement type that changed the interface every client sees, which the agent noted only in a source comment, and a week later 24 theorems still stated as `True` in files that the statement pass had skipped unnoticed, so from then on every file pair was checked.

Round 3. The validator flagged 7 of the 20 pairs with 8 findings, all sent to the agent (Table VI). Two exposed errors that no build could detect: the evaluator lacked two operator classes, and closure typing expanded the namespace relation `nsAll2` by hand without its domain condition. The agent repaired the first and part of the second, and applied four wrong or partly right findings, among them the regrouping of Fig. 3.

Round 4. On the repaired translation the validator flagged 6 pairs with 7 findings. Human experts triaged the findings and escalated three findings in two pairs for adjudication, which rejected all three (Table VI). With every escalated finding rejected, the loop stopped. Triage held back the one correct finding, the sorting comparator, although a human expert had confirmed it, and the reason was not recorded. The stopping rule was therefore met on a translation with a confirmed, unresolved defect: the rule counts only findings that reach adjudication, and this one never did.

Across rounds 3 and 4, four of the 15 findings identified a defect, two of them the same comparator; two flagged the fixed default value of Table II, a difference by design; eight misread HOL4 conventions, such as precedence, finite map updates, and folds; and one misquoted the Lean text.

Rounds 5 and 6. After the proof attempts of Section VI had exposed further defects, round 5 asked whether a newer validator would find them. It ran three times over the 20 pairs of the frozen baseline to show run-to-run variation. All seven defects of Table VIII are in this input and in the round 4 input. The three runs produced 5 findings, all correct (Table V): the `FV` drift that round 3 had introduced and the sorting comparator. The newer model raised no false alarm but missed the other five defects in all runs. Three depend on definitions that the paired prompt does not show (`alist_to_fmap`, `nsAll2`, and Lean's `String.get`), the vacuous statements are well formed, and each admitted equality instance is a `sorry`, which the prompt tells the validator to ignore. To check that the repairs of Section VI-D introduced no new drift, round 6 sent the same prompt to the same model over the repaired translation, three times: all 60 calls answered

TABLE V

REVIEW ROUNDS. FINDINGS: PROBLEMS THE REVIEWER REPORTED, BEFORE ANY CHECK. A CALL REVIEWS ONE FILE PAIR. THE STATE REVIEWED IN ROUND $k \leq 4$ IS THE OUTPUT OF ROUND $k-1$. ROUNDS 5 AND 6 REVIEWED THE FROZEN BASELINE AND ITS REPAIR AFTER THE PROOF STUDY, AND ROUND 7 THE PRELUDE (SECTION VII).

Round	Review input	Findings	Decision and outcome
1 (19 Feb)	Human expert inspection of the pairs	Missing ancestor import and invented interface. Theorems stated as `True` or omitted. Definition bodies replaced by `sorry`. A namespace predicate expanded by hand in some typing rules.	Three repair instructions applied the same day: dependency repair, statement pass, definition audit. The hand expansion remains in the baseline.
2 (26 Feb)	Human expert scan of all pairs	24 theorems still stated as `True`.	All 24 statements restored.
3 (26 Mar)	Validator over 20 pairs, a human expert's reading	13 pairs right, 7 flagged with 8 findings. Human expert: `partial def` still used for 10 functions.	All findings went to the agent without triage. It applied 6 findings in 5 pairs, rejected 2, and replaced every `partial def`.
4 (1 to 8 Apr)	Validator, 20 calls	13 calls answered `RIGHT`, one of them on the wrong files (FpSem was given EvaluateProps), so 12 pairs were right. 6 pairs flagged with 7 findings. The Weakening call failed.	Triage escalated 3 findings in 2 pairs, not the correct one. Adjudication rejected all three, the loop stopped, and the translation was accepted.
5 (5 Oct)	Validator, round 4 prompt, on the frozen baseline, 3 runs of 20 calls	5 findings in 2 pairs, all correct: the free-variable (`FV`) drift in every run, the comparator in two.	A human expert accepted all 5. The agent repaired both defects and five more (Table VIII).
6 (5 Oct)	Same prompt and model on the repaired translation, 3 runs	All 60 calls right, no finding.	No finding, so the loop stopped.
7 (15 Oct)	Validator, round 4 prompt, on 8 prelude sections paired with HOL4 library definitions, 3 runs	9 calls right, 9 distinct findings (Section VII).	A human expert accepted 7: 2 known, 2 documented, 3 new. No repair.

`RIGHT`, so the loop stopped, and the second proving round then exposed two more defects (Section VI-E).

VI. PROOF ATTEMPTS EXPOSED DEFECTS THAT REVIEW MISSED

Proving was planned as a measurement of how much an LLM can prove on fixed statements. It gave three results instead. First, the attempts refuted translated statements and proved vacuous ones trivially, exposing defects that review had missed. Second, on the repaired translation, Aristotle proved the main theorem and 266 further statements unchanged under the strict audit. Third, it refuted two more translated statements, and the library review of Section VII shows that discrepancies remain under the proved theorem. We established each defect by a refutation or a check in Lean and measured every proof with the transitive audit of Section III.

A. Setup

The five workflows are configurations a team might try, from one agent run to more guidance and an automated prover. *Skills* ran Claude Code once with a Lean proving prompt and language server feedback. *Second pass* ran the same prompt and then asked for a second pass over the remaining goals. *Docs* used a prompt that links the Mathlib tactic documentation and suggests scaffolding and strong automation; a human expert reverted its first session, which had added forbidden `partial` definitions, and asked for a second pass. The *Aristotle offered* workflow told Claude Code to hand each file's goals to the Aristotle prover [10] through a Model Context Protocol (MCP) server; the agent never called it. *Direct* gave the whole project to Aristotle alone, on Lean 4.28.0 with Mathlib, in one task and, in the first round, one continuation.

We rebuilt the four Claude Code endpoints with Lean 4.29.1 and the Batteries library and audited every locally complete canonical theorem. We call a statement *tainted* when it depends on `sorryAx`, the axiom behind `sorry`, through the constants it mentions; 120 of the 269 are, 106 of them through the termination proofs of the type variable checks `check_freevars` and `check_freevars_ast`. At every Claude Code endpoint the axiom collection finds no axiom besides the three standard ones and `sorryAx`, so no proof relies on an added axiom or on `native_decide`, which trusts compiled code. The 103 declarations outside the manifest are unchanged at every endpoint, except that Aristotle offered replaced the two admitted instances and Direct moved set declarations to Mathlib.

B. Placeholder counts overstate completion

Placeholder counts overstate completion by up to 29 theorems against the strict audit (Second pass: 104 local, 75 audited) and by up to 12 against the progress measure (Direct, Table VII). Seven local proofs prove the two vacuous statements of Section VI-C, at most two per workflow.

In total the strict audit removes 44 more proofs, all complete modulo the baseline admissions, so here it measures how much of the specification a workflow repaired: only Docs and Aristotle offered, which proved termination of definitions such as `check_freevars` by rewriting them with `List.attach`, have audited proofs of tainted statements.

C. Proofs exposed false and vacuous statements

Of the 55 statement edits over the five endpoints, 50 add `LawfulBEq` assumptions to 28 distinct theorems, 26 of them namespace lemmas. The translation stated these lemmas over an arbitrary `BEq` instance on keys, while HOL4 equality is lawful. Under an instance where no key equals itself, the first conjunct of `nsLookup_nsBind` is false, and we refuted the frozen statement in Lean without axioms. The translated `strsub` indexes bytes, while HOL4 indexes characters, and we refuted the frozen `explode_thm` with `"éa"`, whose

TABLE VI

ALL VALIDATOR FINDINGS OF THE ROUND 3 AND ROUND 4 RUNS, WITH OUR POST HOC CHECK AGAINST THE HOL4 SOURCES. CORRECT: THE LEAN TEXT DIFFERS FROM HOL4 IN MEANING. WRONG: THE TWO AGREE.

#	Rd.	Lean file	Validator claim	Handling	Post hoc check
1	3	Evaluate	Application omits the `Force` and `EvalOp` operator classes	Repaired by agent	Correct
2	3	TypeSoundInvariants	Closure typing rules drop the domain condition of `nsAll2`	Repaired in part by agent	Correct. Value domains now agree, the condition on module paths is still missing in the baseline
3	3	TypeSoundInvariants	Constructor typing rule omits `REVERSE` before building the substitution	Agent added a reversal	Wrong. HOL4's `FEMPTY |++ REVERSE l` keeps the first binding of `l`, as does the baseline `alist_to_fmap`, so the reversal keeps the last
4	3	SemanticPrimitivesProps	Lean should group the `FV` equation of `Letrec` as $A \cup (B \setminus C)$	Agent regrouped	Wrong. `UNION` and `DIFF` are left associative at the same precedence (Fig. 3)
5	3	Weakening	`type_s_weakening` types the store as a list	Agent retyped	Wrong. `store v` abbreviates the same list, so the edit changed nothing
6	3	Mllist	`sort` uses an `Ord` class in place of a comparator	Agent added a comparator returning `ordering`	In part. HOL4 takes a Boolean relation
7	3	SemanticPrimitives	`v_to_char_list` should return a string type	Rejected by agent	Wrong. HOL4 strings are character lists
8	3	TypeSysProps	In `nsLookup_add_tenvE3`, `n` should be a plain name	Rejected by agent	Wrong. `Long` takes a full identifier
9	4	Location	`locnrow` and `locncol` add a default 0	Rejected by human expert	Difference by design (Table II). HOL4 leaves the value unspecified
10	4	Location	Row and column updates return other locations unchanged	Rejected by human expert	Difference by design, as in 9
11	4	SemanticPrimitives	As in 7	Rejected by human expert	Wrong, as in 7
12	4	Misc	`UPDATE_LIST` folds in the wrong direction	Not escalated	Wrong. HOL4 `FOLDL` is a left fold, as is `List.foldl`
13	4	Mllist	The comparator should be Boolean	Not escalated	Correct. Remains in the baseline
14	4	TypeSysProps	As in 8	Not escalated	Wrong, as in 8
15	4	SemanticsMin	`semantics_prog` omits the clock argument	Not escalated	Wrong. The validator misquoted the Lean text, which has the argument

TABLE VII

PROOF COMPLETION ON THE 269 CANONICAL THEOREMS. LOCAL: NO PLACEHOLDER IN THE DECLARATION. MOD.: COMPLETE MODULO THE ADMITTED DEFINITIONS OF THE BASELINE (PROGRESS MEASURE). AUD.: STRICT TRANSITIVE AUDIT (FINAL MEASURE). UNCH.: MOD. WITH UNCHANGED STATEMENT TEXT. S.ED., D.ED.: EDITED STATEMENTS AND DEFINITION BODIES. HELP., MISS.: ADDED AND ABSENT DECLARATIONS.

Workflow	Local	Mod.	Aud.	Unch.	S.ed.	D.ed.	Help.	Miss.
Skills	78	75	69	75	0	0	0	0
Docs	89	89	87	77	12	6	6	0
Second pass	104	98	75	91	7	0	0	0
Aristotle offered	112	112	112	107	5	6	14	0
Direct	120	108	95	81	31	5	2	13
Any	163	151	140					

characters lie in HOL4's range. The other five edits, all in Direct, rename deprecated or clashing names; none of the 55 weakens a statement beyond restoring the HOL4 meaning. The 17 definition edits mostly rewrite recursion with `List.attach` so that termination can be proved. Direct also replaced 11 prelude set definitions with Mathlib's and dropped `decs_type_sound` and the main theorem from the soundness file, so its Miss. column counts these 13.

Proofs by reflexivity exposed vacuous statements: Skills, Docs, and Second pass proved `do_app_cases` by `rfl`. HOL4 computes `do_app_cases` and `type_op_cases` by rewriting the definitions into a case expansion, but the translation states both as $P \leftrightarrow P$. A semantic scan for `True` or for two sides that Lean identifies by light unfolding flags these two and 7 lemmas that are trivial in HOL4 too.

No first-round workflow proved the main theorem `semantics_type_sound`, even locally; 92 of the 118 theorems that none completed are type system, soundness, or weakening lemmas.

D. Repairs

We repaired the seven defects on a branch from the frozen baseline, with Claude Code as the translation agent, and each repair follows the HOL4 text (Table VIII).

The repairs change 3 definitions, 3 constructors, and 43 statements, 41 of them by adding `LawfulBEq` assumptions. Afterwards the project builds, the structural scan finds all 651 obligations, the semantic scan flags only the 7 trivial lemmas, and the only admissions left in definitions are the 13 termination proofs, which taint the same 120 statements as before. Lean proves the repaired `explode_thm`, `nsLookup_nsBind`, `type_op_cases`, and `do_app_cases`, the last modulo termination admissions.

E. Proving the repaired statements exposed two more defects

A repair can itself be wrong, and proving had been the check that exposed the first-round defects, so in October we ran the five workflows again from the repaired translation, each on its own branch, with the first-round messages word for word and the Claude Code runs non-interactive at Claude Code's effort level xhigh.

Table IX gives the counts. The round changed the statements and how the agents were run, so the R1 column shows variation between rounds (Section X). The four Claude Code

TABLE VIII

TRANSLATION DEFECTS IN THE FROZEN BASELINE AND THEIR REPAIRS. #: FINDING IN TABLE VI. R5: ROUND 5 RUNS THAT REPORTED THE DEFECT. MAIN: THE DEFECTIVE CONSTANT IS IN THE CONSTANT CLOSURE OF THE MAIN THEOREM, THE CONSTANTS ITS STATEMENT AND PROOF DEPEND ON.

Defect	Found by	R5	Main	Repair
`FV` of `Letrec` grouped as $A \cup (B \setminus C)$	Post hoc, #4	3/3	no	Parentheses restored
Constructor typing keeps the last binding of a type variable	Post hoc, #3	0/3	yes	`FUPDATE_LIST FEMPTY` over the reversed list, as in HOL4
Closure typing omits the module path condition of `nsAll2`	Post hoc, #2	0/3	yes	Path conditions of both `nsSub` directions added
`sort` takes an `ordering` comparator	#6, #13	2/3	no	Boolean relation, the type of `mergesort_tail`
`strsub` indexes bytes	Direct run, refutation	0/3	yes	Index into the character list, as `EL`
Equality without laws, admitted on values and environments	Proving agents, refutation, scan	0/3	values	Structural `BEq` on values and environments, `LawfulBEq` beside each `BEq` binder
Computed theorems `do_app_cases` and `type_op_cases` stated as $P \leftrightarrow P$	`rfl` proofs, scan	0/3	no	Case expansions that HOL4 computes, 64 and 34 disjuncts

TABLE IX

PROOF COMPLETION ON THE 269 CANONICAL THEOREMS OF THE REPAIRED TRANSLATION, COLUMNS AS IN TABLE VII; NO WORKFLOW LOST A CANONICAL DECLARATION, SO MISS. IS OMITTED. R1: THE FIRST ROUND'S MOD. COUNT. DIRECT'S LOCAL INCLUDES THE 2 RESTATED THEOREMS, AND 13 OF ITS 21 D.ED. ARE REAL EDITS.

Workflow	Local	Mod.	Aud.	Unch.	S.ed.	D.ed.	Help.	R1
Skills	59	59	51	59	0	0	0	75
Docs	63	63	57	63	0	0	0	89
Second pass	132	132	132	132	0	9	0	98
Aristotle offered	71	71	71	71	0	6	0	112
Direct	269	267	267	267	2	21	204	108

workflows added no helper theorems and edited no statements. Aristotle offered rewrote six recursive definitions so that termination can be proved, and Second pass those six and three more. Second pass rose from 98 to 132 theorems modulo admissions, all of them audited, while Skills, Docs, and Aristotle offered fell to 59, 63, and 71. Connecting a prover did not make the agent use it: Aristotle offered never called Aristotle in either round, although its prompt asked for a call on every file and no call was refused, while Direct, which sends every goal to Aristotle, proved 267.

After four continuations, Direct left no `sorry` outside comments. It changed 2 statements and, by our scan, 21 definitions, of which 13 are real edits once the 8 that only gained automatically derived instances are removed, and its proofs of the 267 unchanged statements pass the strict audit, so no termination admission remains. Of these, 266 do not depend on the one definition change that alters meaning, the redefined `CARD`; the two statements and the other definitions are checked below. This endpoint keeps Mathlib as a build dependency but imports no Mathlib module, and Lean's axiom collection over its proofs of `decs_type_sound` and the main theorem `semantics_type_sound` finds only `propext`, `Classical.choice`, and `Quot.sound`.

HOL4, `typeSoundScript`

```
∀(st:'ffi semanticPrimitives$state) env tops r checks
    ctMap tenvS tenv new_tenv tids.
  semantics_prog st env tops r ∧ type_ds checks tenv tops
    tids new_tenv ∧
  type_sound_invariant st env ctMap tenvS tids tenv ⇒ r ≠
    Fail
```

Lean, repaired translation

```
∀ {ffi} (st : cml_state ffi) env tops r checks ctMap_
    tenvS tenv new_tenv tids,
  semantics_prog st env tops r ∧ type_ds checks tenv tops
    tids new_tenv ∧
  type_sound_invariant st env ctMap_ tenvS tids tenv → r
    ≠ .Fail
```

Fig. 4. The main theorem `semantics_type_sound`, Lean binder types elided. In both systems `semantics_prog` relates a program to the behaviour `Fail` exactly when some clock makes `evaluate_prog_with_clock` return `Rerr (Rabort Rtype_error)`.

Both changed statements expose translation defects that review had missed. Aristotle replaced `tlookup_bij_suff` and `tlookup_bij_iff` with versions that assume the keys of the sparse tree are distinct. The prelude represents HOL4's sparse trees as association lists, in which a key can occur twice, while HOL4's trees cannot hold a key twice. For `[(0,1),(1,1),(1,0)]` the values and the keys form the same set, yet lookup maps both keys to 1. We refuted both frozen statements in Lean with only the standard axioms. The added hypothesis is the well-formedness premise of Section II that the representation needed and the translation had left out. By our rule these two proofs stay excluded until the repair is reviewed and accepted, which is still pending.

Of the 13 real definition edits, 12 rewrite recursive definitions, among them the evaluator, so that termination can be proved. The last redefines the prelude's `CARD`, a second prelude error that review missed: the baseline took the length of an arbitrary list enumerating the set, which may contain duplicates, so both `[0]` and `[0,0]` enumerate $\{0\}$ and the definition does not fix the cardinality. Lean proves that the new `CARD` is the length of any duplicate-free enumeration, i.e., the number of elements of a finite set. For the 12 rewritten definitions we have two results of different strength. With only the standard axioms, the rewritten definitions and their mutual partners satisfy their respective original defining equations; the same holds for the rewrites of Second pass and of Aristotle offered. Each rewritten definition also equals a copy of the original, proved by well-founded induction, but this equality rests on the copied original's admitted termination proof.

The invariant is satisfiable. The main theorem would hold trivially if no state satisfied its invariant, and HOL4 rules this out in `prim_type_sound_invariants`, which shows that the initial environment satisfies it. This theorem lies outside the migrated closure, so we translated it afterwards on the Direct endpoint with Claude Code under the same contract.

Aristotle proved it in one task. It changed no statement or definition, and the axiom collection finds only the three standard axioms.

VII. Wrong Definitions Survive a Checked Proof

A proof of the main theorem with only the standard axioms is the strongest evidence the proof stage can give. Type soundness constrains the types of results, not their values, and the proof rests on parts that no proof checks. A review of the library layer found wrong word shifts that reach the main theorem; they are not yet repaired.

What the main theorem trusts. The proof of `semantics_type_sound` uses no admission, but its meaning rests on four things that no proof checks. Its constant closure contains 87 prelude definitions that no review before round 7 paired with a source. The floating point constants use Lean's `Float`, whose operations are opaque to the kernel. The closure also contains 9 of the 12 rewritten definitions, which equal the originals only modulo the originals' admitted termination proofs. Finally, the translation of the 20 theory files was reviewed and scanned but not proved to agree with HOL4. The closure contains neither `CARD` nor the sparse trees.

Round 7. No earlier round had paired the prelude with a source, because it has no HOL4 counterpart file. Round 7 paired it, in 8 sections, with the HOL4 library definitions of the constants it names, fetched from HOL4's `develop` branch on 15 October, in three runs. Of the 24 calls, 9 answered `RIGHT`, and the other 15 reported 9 distinct findings. A human expert checked them against HOL4 and accepted 7, each reported in all three runs. Two are the sparse tree and `CARD` defects above, and two are documented in source comments: `fp64_mul_add` rounds twice, and rounding modes are ignored. Three are new. Sparse tree enumeration follows insertion order, where HOL4 follows the tree. Word shifts by at least the width return the input, where HOL4 returns 0 or the sign fill, and the arithmetic shift by 0 fills with the sign bit. Lean confirms these by evaluation with `decide`, e.g., `word_lsl_8 1 8 = 1`. The two rejected findings come from the split into sections and from the precedence difference of Table II. No run flagged that finite maps drop finiteness, which a source comment documents. The shifts and `fp64_mul_add` reach the main theorem through `do_app`. Lean proves soundness with them, as HOL4 does with the correct ones. Repairing them changes `do_app`, so every proof using it must be rerun.

VIII. Lessons Learned

Each lesson says what another team should do differently, in the order of the workflow stage it concerns.

L1. Treat a compiling translation as a draft of the specification, and check it with a script. Run scripted structural checks after every build and before semantic review, and let them cover instances and vacuous statements such as $P \leftrightarrow P$. Manual checks at milestones let 61 definitions without bodies, among them `do_app`, reach acceptance, although the agent's own README listed the stubs.

L2. Give every finding a recorded, source-grounded decision. The validator found drift that no build could reveal, but in rounds 3 and 4 only four of its 15 findings identified a defect, and wrong findings applied without review became drift. A human decision alone is not enough: triage withheld the one correct finding of round 4 without a recorded reason. Each finding needs a disposition grounded in the HOL4 source, with its reason, and open findings should be tracked. Rejected findings should go back into the validator prompt, because the validator repeated two of them in round 4, and the prompt should treat a `sorry` in a definition differently from one in a proof.

L3. Stop on review coverage and resolved findings, not on the absence of new complaints. Both stops left gaps: in round 4 two pairs had no verdict, and in rounds 4 and 6 the prelude, which held the sparse tree and `CARD` defects, had never been paired with a source. A stopping rule should require a verdict on every pair and a decision on every finding, and should list the definitions in the main theorem's closure that no review has paired with a source, 87 of them before round 7 (Section VII).

L4. Let proof attempts send the translation back to review. Proof attempts exposed six kinds of defect that review had missed (Table I). Defects that proving exposes should go to triage and repair before proofs are counted, and a repair made by a prover while proving needs review like any other. Call a required prover from the workflow rather than offer it to the agent: ours never called the one it was offered.

L5. Measure proofs transitively, and decide which admissions to trust. Report proofs both modulo the admissions you trust and under a strict audit: placeholder counts overstated completion by up to 29 theorems. Trust only admissions that defer work, such as termination proofs, and trust them provisionally, assuming that the admitted definition corresponds to its source, because an admitted instance leaves data unspecified. Compare declarations outside the manifest, such as inductive relations, separately, because no proof count notices a weakened constructor.

Recommended workflow. Together the lessons give the workflow of Fig. 1. Scope the task from the source: record for every source theorem whether it is stated, generated by Lean, or excluded with a reason, and add the theorems that show the main theorem's assumptions satisfiable, which a dependency closure omits (Section VI-E). Run scripted structural checks after every build (L1). Review every file pair, the library layer included, against its source with a second model, and give every finding a source-grounded decision (L2). Before proving starts, the record should hold, for every constant in the main theorem's closure, either its source counterpart or an entry naming the representation difference and the policy that allows it (Table II), and no finding should be open (L3). Then attempt proofs, send what they expose back to review (L4), and count proofs transitively (L5). When a definition or constructor changes, every proof whose closure contains it must be rechecked, and its audit is stale until then. A translation that passes every check is reviewed, not verified,

TABLE X
RELATED WORK. X: BETWEEN PROVERS. H: HOL PROVER. L: LLMS. R: READABLE TARGET. S: VERIFIED SOFTWARE AS SOURCE. M: MECHANICAL FAITHFULNESS CHECK. P: AI PROVES TARGET STATEMENTS. (✓): PARTLY.

Work	X	H	L	R	S	M	P
Proof replay and import [28], [29], [30], [31]	✓	✓				✓	
Babel-formal, Lean2Isabelle [32], [33]	✓		✓	✓			
ITPEval [34]	✓	✓	✓	✓		(✓)	
lf-lean [17]	✓		✓	✓		✓	
CertiCoq, CertiGC, Rust [35], [36], [37]			✓		✓		✓
Sergey [38], [39]			✓				✓
This case	✓	✓	✓	✓	✓		✓

until a mechanical correspondence with the source exists, and its record should say so.

IX. RELATED WORK

Symbolic library exchange (Table X). HOL4 and HOL Light theories have been imported into Isabelle/HOL and Coq by replaying recorded proofs [28], [29], [30], [31], shared through OpenTheory [40], and virtualised inside Isabelle [41]. These produce imported constants or proof objects, not readable source.

LLM translation between provers. Beyond autoformalization of informal mathematics [42], [43], [44], Babel-formal and Lean2Isabelle translate proofs between Lean and Rocq or Isabelle [32], [33], and ITPEval finds library mismatches to be the main obstacle across four provers [34]. lf-lean translates 1,276 statements of Logical Foundations from Rocq to Lean and checks each through a round trip and an isomorphism proof [17]. LLM judges carry systematic biases [45].

Machine learning for proofs. Provers trained with reinforcement learning solve olympiad problems stated in Lean [9], [46], [10], [47]. For verified software, Paraskevopoulou reports on Claude Code writing the Rocq correctness proof of a CertiCoq compiler pass and on the agent silently weakening proof statements [35]. Wang completed the VST proofs of the CertiGC garbage collector with Codex, keeping the decisions on invariants and statements [36]. Sergey mechanised the Move borrow checker in Lean with Claude Code and proved its soundness assumptions satisfiable [38], [39]. Klaus et al. connect Rust code extracted to Lean with AI provers [37]. These works write and review their statements in the target system. A migration adds an obligation of correspondence with an existing formal source, and our evidence shows how representation changes, library substitutions, and wrong review findings violate it while statements stay unchanged and proofs use only the standard axioms. Weakened and vacuous statements, the concerns of [35], [39], are what our proof stage exposed.

X. LIMITATIONS

Measurement. Structural completeness uses textual rules (Section IV), so a `sorry` in one branch of a definition passes them, and the denominator comes from the final target.

Review. The validator is an LLM whose verdicts varied between runs, and rounds 3 and 4 used one run each. In the early rounds, adjudicating findings and operating the agent were not separated, and the decision records of rounds 3 and 4 are incomplete. The post hoc checks of Table VI and the adjudication of round 5 were made by a human expert who by then knew the defects of Table VIII.

Workflow comparison. One run per workflow cannot separate the workflow from run-to-run variation: with the same first message, model, and effort, Skills stopped at 59 theorems while Second pass ran about four times as long over its two passes and reached 132, and between rounds a Claude Code workflow's count moved by up to 41 theorems.

Generality. We studied one development and one pair of provers. Each state mixes prompts, model output, and human repair, so no intervention is isolated. CakeML is public, so the models may have seen it in training.

XI. CONCLUSION

In this migration of CakeML's type soundness theorem from HOL4 to Lean, a checked proof of the main theorem coexists with library operations that still differ from HOL4, because type soundness does not constrain the values that operations compute. The workflow we recommend assigns each claim about a translation to a check that can support it and sends what review and proving expose back to repair.

Future work. We plan to apply the recommended workflow from scratch to the same development, with several models and repeated runs at matched budgets; to repair the word shifts and fused multiply-add of Section VII and the sparse tree representation of Section VI-E, and rerun the proofs they reach; to run both evaluators on the same programs; and to migrate a second development.

Artifact. The artifact contains the reduced HOL4 package, the translation at every milestone of Table IV, the baseline, the ten proving endpoints, the repaired translation, the validator records of rounds 3 to 7, the session logs, and the analysis scripts. Its scripts recompute the reported numbers, and its README says which results are rerun and which are recorded, and lists computers, tool versions, and costs. Repository links are withheld for anonymous review. We submit the reproduction package to the artifact track.

ACKNOWLEDGEMENTS

Acknowledgements are omitted for anonymous review.

Use of generative AI. Besides the models reported above, Claude and ChatGPT helped refine the text and the analysis scripts.

REFERENCES

[1] K. Slind and M. Norrish, "A brief overview of HOL4," in *Theorem Proving in Higher Order Logics (TPHOLs)*, ser. Lecture Notes in Computer Science, vol. 5170. Springer, 2008, pp. 28–32.

[2] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, ser. Lecture Notes in Computer Science. Springer, 2002, vol. 2283.

[3] Y. Bertot and P. Castéran, *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*, ser. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004.

[4] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction (CADE-28)*, ser. Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635.

[5] M. J. Gordon, R. Milner, and C. P. Wadsworth, *Edinburgh LCF: A Mechanised Logic of Computation*, ser. Lecture Notes in Computer Science. Springer, 1979, vol. 78.

[6] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens, "CakeML: A verified implementation of ML," in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2014, pp. 179–191.

[7] T. Ringer, K. Palmskog, I. Sergey, M. Gligoric, and Z. Tatlock, "QED at large: A survey of engineering of formally verified software," *Foundations and Trends in Programming Languages*, vol. 5, no. 2–3, pp. 102–281, 2019.

[8] The mathlib Community, "The Lean mathematical library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. ACM, 2020, pp. 367–381.

[9] Z. Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang *et al.*, "DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition," arXiv:2504.21801, 2025.

[10] T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico *et al.*, "Aristotle: IMO-level automated theorem proving," arXiv:2510.01346, 2025.

[11] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sankar, L. P. Wassi, M. Merler, B. Sobolev, R. Pavuluri, S. Sinha, and R. Jabbarvand, "Lost in translation: A study of bugs introduced by large language models while translating code," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2024.

[12] C. Ziftci, S. Nikolov, A. Sjövall, B. Kim, D. Codecasa, and M. Kim, "Migrating code at scale with LLMs at Google," in *ACM International Conference on the Foundations of Software Engineering (FSE), Industry Papers*, 2025, arXiv:2504.09691.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[14] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 50 528–50 652.

[15] Anthropic, "Claude Code overview," https://code.claude.com/docs/en/overview, 2026, accessed October 2026.

[16] Y. K. Tan, S. Owens, and R. Kumar, "A verified type system for CakeML," in *Proceedings of the 27th Symposium on the Implementation and Application of Functional Programming Languages (IFL)*. ACM, 2015, pp. 7:1–7:12.

[17] V. Saraswat, J. Chang, H. Agrawal, V. Agarwalla, R. Zhang, T. Stone, J. Green, S. Vaahan, L. Chan, R. Agrawal, and J. Gross, "lf-lean: The frontier of verified software engineering," Blog post, https://theorem.dev/blog/lf-lean/, 2026, published February 5, 2026.

[18] Anonymous, "Master's dissertation," Details withheld for double-anonymous review, 2026.

[19] Y. K. Tan, M. O. Myreen, R. Kumar, A. Fox, S. Owens, and M. Norrish, "The verified CakeML compiler backend," *Journal of Functional Programming*, vol. 29, p. e2, 2019.

[20] S. Owens, M. O. Myreen, R. Kumar, and Y. K. Tan, "Functional big-step semantics," in *Programming Languages and Systems (ESOP)*, ser. Lecture Notes in Computer Science, vol. 9632. Springer, 2016, pp. 589–615.

[21] K. Slind, "Function definition in higher-order logic," in *Theorem Proving in Higher Order Logics (TPHOLs)*, ser. Lecture Notes in Computer Science, vol. 1125. Springer, 1996, pp. 381–397.

[22] Lean Development Team, "The Lean language reference: Partial and unsafe definitions," https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/, accessed October 2026.

[23] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.

[24] Gemini Team, Google, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," arXiv:2507.06261, 2025.

[25] Google DeepMind, "Gemini 3.8 Flash model card," https://deepmind.google/models/model-cards/gemini-3-8-flash/, 2026, accessed October 2026.

[26] Artificial Analysis, "Model pages for Claude Haiku 5.5 and Gemini 3.8 Flash (High)," https://artificialanalysis.ai/models/claude-haiku-5-5, https://artificialanalysis.ai/models/gemini-3-8-flash, 2026, accessed 23 October 2026.

[27] Lean Development Team, "The Lean language reference: Axioms," https://lean-lang.org/doc/reference/latest/Axioms/, accessed October 2026.

[28] S. Obua and S. Skalberg, "Importing HOL into Isabelle/HOL," in *Automated Reasoning (IJCAR)*, ser. Lecture Notes in Computer Science, vol. 4130. Springer, 2006, pp. 298–302.

[29] C. Keller and B. Werner, "Importing HOL Light into Coq," in *Interactive Theorem Proving (ITP)*, ser. Lecture Notes in Computer Science, vol. 6172. Springer, 2010, pp. 307–322.

[30] C. Kaliszyk and A. Krauss, "Scalable LCF-style proof translation," in *Interactive Theorem Proving (ITP)*, ser. Lecture Notes in Computer Science, vol. 7998. Springer, 2013, pp. 51–66.

[31] F. Blanqui, "Translating HOL-Light proofs to Coq," in *Proceedings of the 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning (LPAR)*, ser. EPiC Series in Computing, vol. 100. EasyChair, 2024, pp. 1–18.

[32] T. Stoskopf, C. Cohen, and N. Tabareau, "Babel-formal: Translation of proofs between Lean and Rocq," in *MATH-AI: The 5th Workshop on Mathematical Reasoning and AI at NeurIPS*, 2025.

[33] G. Liu, J. Ma, and D. F. Wong, "Lean2Isabelle: Factorized cross-assistant proof translation," in *3rd AI for Math Workshop at ICML*, 2026.

[34] J. Wu, R. J. George, and A. Anandkumar, "ITPEval: Benchmarking formal translation across interactive theorem provers," in *3rd AI for Math Workshop at ICML*, 2026, arXiv:2607.19407.

[35] Z. Paraskevopoulou, "Machine-generated, machine-checked proofs for a verified compiler (experience report)," *Proceedings of the ACM on Programming Languages*, vol. 10, no. ICFP, pp. 807–826, 2026.

[36] S. Wang, "AI-assisted completion of CertiGC proofs: An experience report," arXiv:2606.21167, 2026.

[37] N. Klaus, J. Conejero, and P. Tolmach, "A Rust-to-Lean verification pipeline with AI provers: An experience report," in *AIMACS Workshop at CAV*, 2026, arXiv:2605.30106.

[38] T. Nowacki, S. Blackshear, J. Mitchell, S. Qadeer, and I. Sergey, "Tracking borrows with regular expressions," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA2, pp. 389:1–389:27, 2026.

[39] I. Sergey, "Verifying Move borrow checker in Lean: An experiment in AI-assisted PL metatheory," https://proofsandintuitions.net/2026/03/18/move-borrow-checker-lean/, 2026, blog post, accessed October 2026.

[40] J. Hurd, "The OpenTheory standard theory library," in *NASA Formal Methods (NFM)*, ser. Lecture Notes in Computer Science, vol. 6617. Springer, 2011, pp. 177–191.

[41] F. Immler, J. Rädle, and M. Wenzel, "Virtualization of HOL4 in Isabelle," in *10th International Conference on Interactive Theorem Proving (ITP)*, ser. LIPIcs, vol. 141. Schloss Dagstuhl, 2019, pp. 21:1–21:18.

[42] Y. Wu, A. Q. Jiang, W. Li, M. N. Rabe, C. Staats, M. Jamnik, and C. Szegedy, "Autoformalization with large language models," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.

[43] A. Q. Jiang, S. Welleck, J. P. Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y. Wu, and G. Lample, "Draft, sketch, and prove: Guiding formal theorem provers with informal proofs," in *International Conference on Learning Representations (ICLR)*, 2023.

[44] K. Weng, L. Du, S. Li, W. Lu, H. Sun, H. Liu, and T. Zhang, "Autoformalization in the era of large language models: A survey," arXiv:2505.23486, 2025.

[45] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023, pp. 46 595–46 623.

[46] T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, G. Žužić, E. Wieser, A. Huang, J. Schrittwieser, Y. Schroecker, H. Masoom *et al.*, "Olympiad-level formal mathematical reasoning with reinforcement learning," *Nature*, vol. 651, no. 8106, pp. 607–613, 2026.

[47] K. Zheng, J. M. Han, and S. Polu, "miniF2F: A cross-system benchmark for formal olympiad-level mathematics," in *International Conference on Learning Representations (ICLR)*, 2022.